Calculating Powerlifting Scores as a Professional Coach

Dynamic Objective Team Scoring (Dots)

In the world of powerlifting, comparing lifters across different body weights is essential to /determine relative strength and fairness in competition. This is where the **DOTS score**—short for “Dynamic Objective Team Scoring”—comes into play. It’s a widely-used formula that helps level the playing field by standardizing performances regardless of a lifter’s body weight. Whether you’re new to powerlifting or a seasoned competitor, understanding the DOTS score is crucial for evaluating progress and competing effectively.
This section is from [powerliftpro](https://powerliftpro.app/understanding-the-dots-score-in-powerlifting-a-comprehensive-guide/).

What is the DOTS Score?

The DOTS score is a mathematical formula used to normalize powerlifting totals based on a lifter’s body weight. It provides a single number that represents a lifter’s relative strength, allowing for fair comparisons across all weight classes.

The score takes into account:

- **The lifter's total:** The combined weight lifted in the [squat](https://powerliftpro.app/exercises/low-bar-squat/), [bench press](https://powerliftpro.app/exercises/comp-bench-press/), and [deadlift](https://powerliftpro.app/exercises/conventional-deadlift/).
- **Body weight:** The lifter’s weight on competition day.

The DOTS Formula

For real powerlifting nerds who want to calculate their DOTS longhand (remember to show all work! sorry, old math class joke), the DOTS formula is as follows:

DOTS Score=Total Weight Lifted (kg)×a+b(BW)+c(BW2)+d(BW3)+e(BW4)500

Here:

- **Total Weight Lifted (kg):** Your combined total from the squat, bench press, and deadlift.
- **BW:** Your body weight in kilograms.
- **a, b, c, d, e:** Coefficients derived from statistical modeling to ensure accurate scaling across various body weights.

These coefficients are carefully designed to balance the advantage heavier lifters might have in absolute strength and the lighter lifters’ advantage in relative strength.

```rust
use crate::poly4;
use opltypes::*;

pub fn dots_coefficient_men(bodyweightkg: f64) -> f64 {
    const A: f64 = -0.0000010930;
    const B: f64 = 0.0007391293;
    const C: f64 = -0.1918759221;
    const D: f64 = 24.0900756;
    const E: f64 = -307.75076;

    // Bodyweight bounds are defined; bodyweights out of range match the boundaries.
    let adjusted = bodyweightkg.clamp(40.0, 210.0);
    500.0 / poly4(A, B, C, D, E, adjusted)
}

pub fn dots_coefficient_women(bodyweightkg: f64) -> f64 {
    const A: f64 = -0.0000010706;
    const B: f64 = 0.0005158568;
    const C: f64 = -0.1126655495;
```

```
    const D: f64 = 13.6175032;
    const E: f64 = -57.96288;

    // Bodyweight bounds are defined; bodyweights out of range match the boundaries.
    let adjusted = bodyweightkg.clamp(40.0, 150.0);
    500.0 / poly4(A, B, C, D, E, adjusted)
}

/// Calculates Dots points.
///
/// Dots were introduced by the German IPF Affiliate BVDK after the IPF switched to
/// IPF Points, which do not allow comparing between sexes. The BVDK hosts team
/// competitions that allow lifters of all sexes to compete on a singular team.
///
/// Since Wilks points have been ostracized from the IPF, and IPF Points are
/// unsuitable, German lifters therefore came up with their own formula.
///
/// The author of the Dots formula is Tim Konertz <tim.konertz@outlook.com>.
///
/// Tim says that Dots is an acronym for "Dynamic Objective Team Scoring,"
/// but that they chose the acronym before figuring out the expansion.
pub fn dots(sex: Sex, bodyweight: WeightKg, total: WeightKg) -> Points {
    if bodyweight.is_zero() || total.is_zero() {
        return Points::from_i32(0);
    }
    let coefficient: f64 = match sex {
        Sex::M | Sex::Mx => dots_coefficient_men(f64::from(bodyweight)),
        Sex::F => dots_coefficient_women(f64::from(bodyweight)),
    };
    Points::from(coefficient * f64::from(total))
}
```

IPF Good Lift Coefficient

The **IPF Good Lift Coefficient** calculator computes the comparative weight coefficient between weight lifters based on the weight of the lifter (x), the type of lift and the gender of the lifter, all combined in the IPF GL Coefficient formula. Information about this is from [IPF](https://www.powerlifting.sport/fileadmin/ipf/data/ipf-formula/IPF_GL_Coefficients-2020.pdf).

INSTRUCTIONS: Choose units and enter the following:

- (**x**) Weigh of Person
- (**g**) Gender of Person
- (LT) lift type
  - Equipped Power Lift
  - Classic Power Lift
  - Equipped Bench Press
  - Classic Bench Press

IPFL GL Coefficient (IPC): The calculator returns the coefficient as a real number (decimal).

The Math / Science

The International Powerlifting Federation GL coefficient formula is:

$$IPC = 100A - B \cdot e^{-C \cdot BWT}$$

where:

- IPC = IPF GL coefficient
- BWT = body weight of the lifter
- A, B, C = f(gender, lift type), see below

```rust
```

```
use opltypes::*;

/// Hardcoded formula parameters: `(A, B, C)`.
type Parameters = (f64, f64, f64);

/// Gets formula parameters from what is effectively a lookup table.
fn parameters(sex: Sex, equipment: Equipment, event: Event) -> Parameters {
    // Since the formula was made for the IPF, it only covers Raw and Single-ply.
    // We do our best and just reuse those for Wraps and Multi-ply, respectively.
    let equipment = match equipment {
        Equipment::Raw | Equipment::Wraps | Equipment::Straps => Equipment::Raw,
        Equipment::Single | Equipment::Multi | Equipment::Unlimited => Equipment::Single,
    };

    // Points are only specified for Sex::M and Sex::F.
    let dichotomous_sex = match sex {
        Sex::M | Sex::Mx => Sex::M,
        Sex::F => Sex::F,
    };

    const SBD: Event = Event::sbd();
    const B: Event = Event::b();

    match (event, dichotomous_sex, equipment) {
        (SBD, Sex::M, Equipment::Raw) => (1199.72839, 1025.18162, 0.009210),
        (SBD, Sex::M, Equipment::Single) => (1236.25115, 1449.21864, 0.01644),
        (SBD, Sex::F, Equipment::Raw) => (610.32796, 1045.59282, 0.03048),
        (SBD, Sex::F, Equipment::Single) => (758.63878, 949.31382, 0.02435),

        (B, Sex::M, Equipment::Raw) => (320.98041, 281.40258, 0.01008),
        (B, Sex::M, Equipment::Single) => (381.22073, 733.79378, 0.02398),
        (B, Sex::F, Equipment::Raw) => (142.40398, 442.52671, 0.04724),
        (B, Sex::F, Equipment::Single) => (221.82209, 357.00377, 0.02937),

        _ => (0.0, 0.0, 0.0),
    }
}

/// Calculates IPF GOODLIFT Points.
pub fn goodlift(
    sex: Sex,
    equipment: Equipment,
    event: Event,
    bodyweight: WeightKg,
    total: WeightKg,
) -> Points {
    // Look up parameters.
    let (a, b, c) = parameters(sex, equipment, event);

    // Exit early for undefined cases.
    if a == 0.0 || bodyweight < WeightKg::from_i32(35) || total.is_zero() {
        return Points::from_i32(0);
    }

    // A - B * e^(-C * Bwt).
    let e_pow = (-c * f64::from(bodyweight)).exp();
    let denominator = a - (b * e_pow);

    // Prevent division by zero.
    if denominator == 0.0 {
        return Points::from_i32(0);
    }

    // Calculate GOODLIFT points.
    // We add the requirement that the value be non-negative.
```

```
    let points: f64 = f64::from(total) * (0.0_f64).max(100.0 / denominator);
    Points::from(points)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn published_examples() {
        // Dmitry Inzarkin from 2019 IPF World Open Men's Championships.
        let weight = WeightKg::from_f32(92.04);
        let total = WeightKg::from_f32(1035.0);
        assert_eq!(
            goodlift(Sex::M, Equipment::Single, Event::sbd(), weight, total),
            Points::from(112.85)
        );

        // Susanna Torronen from 2019 World Open Classic Bench Press Championships.
        let weight = WeightKg::from_f32(70.50);
        let total = WeightKg::from_f32(122.5);
        assert_eq!(
            goodlift(Sex::F, Equipment::Raw, Event::b(), weight, total),
            Points::from(96.78)
        );
    }
}
```

Wilks coefficient

The following [equation](https://en.wikipedia.org/wiki/Equation) is used to calculate the Wilks coefficient:
$$
\text{Coef} = \frac{500}{a + bx + cx^2 + dx^3 + ex^4 + fx^5}
$$
where x is the body weightof the lifter in kilograms.

The total weight lifted (in kg) is multiplied by the coefficient to find the standard amount lifted, normalised across all body weights.

	Men	*Women*
a	-216.0475144	594.31747775582
b	16.2606339	−27.23842536447
c	-0.002388645	0.82112226871
d	-0.00113732	−0.00930733913
e	7.01863 × 10−6	4.731582 × 10−5
f	−1.291 × 10−8	−9.054 × 10−8

```rust
use crate::poly5;
use opltypes::*;

pub fn wilks_coefficient_men(bodyweightkg: f64) -> f64 {
    // Wilks defines its polynomial backwards:
    // A + Bx + Cx^2 + ...
    const A: f64 = -216.0475144;
    const B: f64 = 16.2606339;
    const C: f64 = -0.002388645;
    const D: f64 = -0.00113732;
    const E: f64 = 7.01863E-06;
    const F: f64 = -1.291E-08;
```

```
    // Upper bound avoids asymptote.
    // Lower bound avoids children with huge coefficients.
    let adjusted = bodyweightkg.clamp(40.0, 201.9);

    500.0 / poly5(F, E, D, C, B, A, adjusted)
}

pub fn wilks_coefficient_women(bodyweightkg: f64) -> f64 {
    const A: f64 = 594.31747775582;
    const B: f64 = -27.23842536447;
    const C: f64 = 0.82112226871;
    const D: f64 = -0.00930733913;
    const E: f64 = 0.00004731582;
    const F: f64 = -0.00000009054;

    // Upper bound avoids asymptote.
    // Lower bound avoids children with huge coefficients.
    let adjusted = bodyweightkg.clamp(26.51, 154.53);

    500.0 / poly5(F, E, D, C, B, A, adjusted)
}

/// Calculates Wilks points.
pub fn wilks(sex: Sex, bodyweight: WeightKg, total: WeightKg) -> Points {
    if bodyweight.is_zero() || total.is_zero() {
        return Points::from_i32(0);
    }
    let coefficient: f64 = match sex {
        Sex::M | Sex::Mx => wilks_coefficient_men(f64::from(bodyweight)),
        Sex::F => wilks_coefficient_women(f64::from(bodyweight)),
    };
    Points::from(coefficient * f64::from(total))
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn coefficients() {
        // Coefficients taken verbatim from the old Python implementation.
        assert_eq!(wilks_coefficient_men(100.0), 0.6085890719066511);
        assert_eq!(wilks_coefficient_women(100.0), 0.8325833167368228);
    }

    #[test]
    fn points() {
        // Point values taken (rounded) from the old Python implementation.
        assert_eq!(
            wilks(Sex::M, WeightKg::from_i32(100), WeightKg::from_i32(1000)),
            Points::from(608.58907)
        );
        assert_eq!(
            wilks(Sex::F, WeightKg::from_i32(60), WeightKg::from_i32(500)),
            Points::from(557.4434)
        );
    }
}
```

Glossbrenner

When Powerlifting began having 'masters', it quickly became apparent that some sort of age allowance or 'handicap' was needed by this group to sort them out, one from another. How much better must a

40-yr-old be, than a 50-yr-old, to be considered a better lifter? How much better than a 55-yr-old, a 60-yr-old, etc? So, a set of age multipliers, or 'handicap numbers', was issued for Master Lifters year-by-year, from 40 through 80. Since then, however, the number of Master Lifters in the 50+ and 60+ group has become so large that a revision (based on extensive study and years of backup data) was needed. These new numbers are based on comparison of the totals of actual lifters over the past seven years, and a chart for their usage is on the final page of this [section](https://worldpowerliftingcongress.com/wp-content/uploads/2015/02/Glossbrenner.htm).

The formula is: (LT) times (GBC) = (PN)

LT - Lifter's Total
GBC – Glossbrenner Bodyweight Coefficient *
PN - Product Number

```rust
use opltypes::*;

use crate::schwartzmalone::{malone_coefficient, schwartz_coefficient};
use crate::wilks::{wilks_coefficient_men, wilks_coefficient_women};

fn glossbrenner_coefficient_men(bodyweightkg: f64) -> f64 {
    // Glossbrenner is defined piecewise.
    if bodyweightkg < 153.05 {
        (schwartz_coefficient(bodyweightkg) + wilks_coefficient_men(bodyweightkg)) / 2.0
    } else {
        // Linear coefficients found by fitting to a table.
        const A: f64 = -0.000821668402557;
        const B: f64 = 0.676940740094416;
        (schwartz_coefficient(bodyweightkg) + A * bodyweightkg + B) / 2.0
    }
}

fn glossbrenner_coefficient_women(bodyweightkg: f64) -> f64 {
    // Glossbrenner is defined piecewise.
    if bodyweightkg < 106.3 {
        (malone_coefficient(bodyweightkg) + wilks_coefficient_women(bodyweightkg)) / 2.0
    } else {
        // Linear coefficients found by fitting to a table.
        const A: f64 = -0.000313738002024;
        const B: f64 = 0.852664892884785;
        (malone_coefficient(bodyweightkg) + A * bodyweightkg + B) / 2.0
    }
}

/// Calculates Glossbrenner points.
///
/// Glossbrenner is the average of two older systems, Schwartz-Malone and Wilks,
/// with a piecewise linear section.
///
/// This points system is most often used by GPC affiliates.
pub fn glossbrenner(sex: Sex, bodyweight: WeightKg, total: WeightKg) -> Points {
    if bodyweight.is_zero() || total.is_zero() {
        return Points::from_i32(0);
    }
    let coefficient: f64 = match sex {
        Sex::M | Sex::Mx => glossbrenner_coefficient_men(f64::from(bodyweight)),
        Sex::F => glossbrenner_coefficient_women(f64::from(bodyweight)),
    };
    Points::from(coefficient * f64::from(total))
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
```

```
    fn coefficients() {
        // Coefficients taken verbatim from the old Python implementation.
        assert_eq!(glossbrenner_coefficient_men(100.0), 0.5812707859533183);
        assert_eq!(glossbrenner_coefficient_women(100.0), 0.7152488066040259);
    }

    #[test]
    fn points() {
        // Point values taken (rounded) from the old Python implementation.
        assert_eq!(
            glossbrenner(Sex::M, WeightKg::from_i32(100), WeightKg::from_i32(1000)),
            Points::from(581.27)
        );
        assert_eq!(
            glossbrenner(Sex::F, WeightKg::from_i32(60), WeightKg::from_i32(500)),
            Points::from(492.53032)
        );

        // Zero bodyweight should be treated as "unknown bodyweight".
        assert_eq!(
            glossbrenner(Sex::M, WeightKg::from_i32(0), WeightKg::from_i32(500)),
            Points::from_i32(0)
        );
    }
}
```